

Data: _____

RACCOLTA PROVE D'ESAME: DESIGN PATTERN E PROGRAMMAZIONE FUNZIONALE (VOL. 2)

Il documento contiene 10 prove simulate, ciascuna composta da 2 esercizi, per un totale di 20 esercizi.

Nome: _____

Matr.: _____

Prova 1

1. Per il seguente codice relativo a un processo strutturato:

```
abstract class ProcessoDati {
    public void esegui() {
        leggiDati();
        elaboraDati();
        salvaDati();
    }
    abstract void leggiDati();
    abstract void elaboraDati();
    void salvaDati() { /* salvataggio generico */ }
}
class ProcessoCSV extends ProcessoDati { ... }
```

1. indicare il design pattern ed i nomi dei ruoli di ciascuna classe fornita,
2. implementare il codice di **ProcessoCSV** e di un'altra classe **ProcessoDatabase** sovrascrivendo i metodi necessari,
3. disegnare il diagramma UML delle classi fornite e create (indicando nomi classi, nomi metodi, e relazioni),
4. disegnare il diagramma UML di sequenza in cui l'esecuzione del metodo **esegui()** viene avviata da un Client su un'istanza di **ProcessoCSV**.

2. Usando la programmazione funzionale di Java, data una lista **ricettario** di istanze di **Ricetta**, implementare una classe con:

```
public record Ricetta(String nome, String categoria, int tempoPrep, int calorie)
{ }
```

1. un metodo che trova le ricette con **tempoPrep** inferiore a un parametro e restituisce la lista dei loro nomi,
2. un metodo che conta il numero di categorie diverse presenti nella lista,
3. un metodo che calcola la somma delle **calorie** di tutte le ricette della categoria "Dolce",
4. un metodo che restituisce l'istanza di **Ricetta** con il maggior numero di calorie in assoluto.

Nome: _____

Matr.: _____

Prova 2

1. Per il seguente codice relativo all'esecuzione di richieste:

```
interface Comando {
    void esegui();
}
class Luce {
    void accendi() { ... }
    void spegni() { ... }
}
class ComandoAccendi implements Comando {
    Luce luce;
    ...
}
class Telecomando {
    Comando slot;
    void premiPulsante() { ... }
}
```

1. indicare il design pattern ed i ruoli delle interfacce e classi fornite,
2. completare l'implementazione del pattern aggiungendo il costruttore ed il metodo `esegui()` in `ComandoAccendi` e completando `Telecomando`,
3. implementare un `Client` che configura il telecomando con un comando per accendere una luce e preme il pulsante,
4. disegnare il diagramma UML delle classi (indicando nomi classi, metodi e relazioni) e il diagramma UML di sequenza a partire dal `Client`.

2. Usando la programmazione funzionale di Java, data una lista `store` di istanze di `App`, implementare una classe con:

```
public record App(String nome, String sviluppatore, double prezzo, int download)
{ }
```

1. un metodo che restituisce la lista dei nomi delle app sviluppate da uno `sviluppatore` passato come parametro,
2. un metodo che restituisce una lista di stringhe con i nomi degli sviluppatori, assicurandosi che non ci siano duplicati,
3. un metodo che restituisce la somma dei `download` di tutte le app gratuite (con `prezzo` uguale a 0),
4. un metodo che restituisce il `nome` dell'app che ha il prezzo più alto.

Nome:

Matr.:

Prova 3

1. Per il seguente codice per la creazione di veicoli:

```
interface Veicolo {
    void avvia();
}
class Auto implements Veicolo { ... }
class Moto implements Veicolo { ... }
abstract class VeicoloFactory {
    abstract Veicolo creaVeicolo();
}
class AutoFactory extends VeicoloFactory { ... }
```

1. indicare il design pattern ed i nomi dei ruoli di ciascuna interfaccia e classe,
2. completare l'implementazione del pattern con la classe **AutoFactory** e implementare una **MotoFactory**,
3. implementare una classe **Client** che utilizza la factory appropriata per istanziare e avviare una moto,
4. disegnare il diagramma UML delle classi fornite e implementate, compreso il **Client**,
5. disegnare il diagramma UML di sequenza in cui l'esecuzione inizia dalla classe **Client**.

2. Usando la programmazione funzionale di Java, data una lista **voli** di istanze di **Volo**, implementare una classe con:

```
public record Volo(String codice, String compagnia, int durataMinuti, int passeggeri) { }
```

1. un metodo che trova nella lista i voli gestiti da una determinata **compagnia** passata in input e restituisce la lista dei loro codici,
2. un metodo che conta il numero di compagnie aeree diverse presenti nella lista e lo restituisce,
3. un metodo che calcola la media della **durataMinuti** di tutti i voli della lista,
4. un metodo che restituisce l'istanza di **Volo** con il minor numero di passeggeri.

Nome:

Matr.:

Prova 4

1. Per il seguente codice relativo a una catena di gestione del supporto:

```
abstract class GestoreSupporto {
    GestoreSupporto successivo;
    public void setSuccessivo(GestoreSupporto s) { ... }
    public abstract void gestisciRichiesta(int livello);
}
class SupportoBase extends GestoreSupporto { ... }
class SupportoAvanzato extends GestoreSupporto { ... }
```

1. indicare il design pattern utilizzato ed i ruoli delle classi fornite,
2. completare l'implementazione in modo che **SupportoBase** gestisca richieste di livello ≤ 1 e passi le altre al successivo, e **SupportoAvanzato** gestisca i livelli superiori,
3. implementare un **Client** che crea la catena e invia una richiesta di livello 2,

- disegnare il diagramma UML delle classi fornite e create, e della classe `Client`,
- disegnare il diagramma UML di sequenza a partire dal `Client` per la richiesta di livello 2.

2. Usando la programmazione funzionale di Java, data una lista `catalogo` di istanze di `Immobile`, implementare una classe con:

```
public record Immobile(String id, String citta, double mq, double prezzo) { }
```

- un metodo che restituisce la lista degli `id` degli immobili con una superficie (`mq`) maggiore di un valore in input,
- un metodo che conta il numero di `citta` diverse presenti nel catalogo,
- un metodo che restituisce la somma dei `prezzo` di tutti gli immobili situati nella città "Roma",
- un metodo che restituisce l'istanza di `Immobile` con il rapporto `prezzo/mq` più basso.

Nome:

Matr.:

Prova 5

1. Per il seguente codice relativo a una configurazione globale:

```
class ConfigurazioneSistema {  
    private String tema = "Scuro";  
    // istanza e metodi necessari mancanti  
    public String getTema() { return tema; }  
}
```

- indicare il design pattern che assicura la presenza di una sola istanza della classe,
- completare l'implementazione del pattern aggiungendo il costruttore appropriato, l'attributo statico e il metodo per ottenere l'istanza,
- implementare una classe `Client` che ottiene l'istanza e stampa a schermo il tema,
- disegnare il diagramma UML delle classi (indicando nomi, metodi e modificatori di accesso),
- disegnare il diagramma UML di sequenza in cui l'esecuzione inizia dal `Client`.

2. Usando la programmazione funzionale di Java, data una lista `archivio` di istanze di `Contratto`, implementare una classe con:

```
public record Contratto(String numero, String tipo, int mesi, double valore) { }
```

- un metodo che restituisce la lista dei `numero` dei contratti che hanno un `tipo` uguale a un parametro in ingresso,
- un metodo che restituisce la lista dei tipi di contratto presenti, senza duplicati,
- un metodo che calcola la somma dei `valore` di tutti i contratti,
- un metodo che restituisce l'istanza del contratto con la durata in `mesi` maggiore in assoluto.

Nome:

Matr.:

Prova 6

1. Per il seguente codice che disaccoppia un'astrazione dalla sua implementazione:

```
interface Dispositivo {
    void accendi();
    void spegni();
}
class TV implements Dispositivo { ... }
abstract class Telecomando {
    protected Dispositivo disp;
    public Telecomando(Dispositivo d) { this.disp = d; }
    public abstract void premiPower();
}
class TelecomandoAvanzato extends Telecomando {
    public void premiMuto() { ... }
    ...
}
```

1. indicare il design pattern ed i ruoli delle astrazioni e implementazioni,
 2. completare l'implementazione di `TelecomandoAvanzato` e creare un `TelecomandoBase`, implementando i rispettivi metodi,
 3. implementare una classe `Client` che associa una `TV` a un `TelecomandoAvanzato` e lo accende,
 4. disegnare il diagramma UML delle classi fornite, create e del `Client`,
 5. disegnare il diagramma UML di sequenza a partire dall'invocazione sul `Client`.
2. Usando la programmazione funzionale di Java, data una lista `libretto` di istanze di `Esame`, implementare una classe con:

```
public record Esame(String materia, String studente, int voto, int crediti) { }
```

1. un metodo che restituisce la lista delle `materia` per cui il `voto` è maggiore o uguale a 18,
2. un metodo che conta il numero di studenti diversi presenti nel libretto,
3. un metodo che calcola la somma dei `credit`i degli esami passati (`voto >= 18`),
4. un metodo che restituisce l'istanza di `Esame` con il voto più alto registrato.

Nome:

Matr.:

Prova 7

1. Per il seguente codice riguardante famiglie di prodotti:

```
interface Sedia { boolean haGambe(); }
interface Divano { int posti(); }
```

```
interface MobiliFactory {
    Sedia creaSedia();
    Divano creaDivano();
}
class SediaModerna implements Sedia { ... }
class DivanoModerno implements Divano { ... }
class FactoryModerna implements MobiliFactory { ... }
```

1. indicare il design pattern utilizzato ed i ruoli delle interfacce e classi,
2. completare l'implementazione di **FactoryModerna** per restituire le rispettive istanze,
3. implementare una classe **Client** che istanzia la **FactoryModerna** e la usa per creare una sedia e un divano,
4. disegnare il diagramma UML delle classi indicando le relazioni di dipendenza e realizzazione,
5. disegnare il diagramma UML di sequenza del **Client** che crea i due oggetti.

2. Usando la programmazione funzionale di Java, data una lista **stock** di istanze di **Smartphone**, implementare una classe con:

```
public record Smartphone(String modello, String marca, int ram, double prezzo) { }
```

1. un metodo che trova gli smartphone di una data **marca** in input e restituisce la lista dei loro modelli,
2. un metodo che restituisce la lista delle marche presenti, senza duplicati,
3. un metodo che calcola il prezzo medio di tutti gli smartphone presenti nello stock,
4. un metodo che restituisce lo smartphone con la dotazione di **ram** maggiore.

Nome:

Matr.:

Prova 8

1. Per il seguente codice per il coordinamento delle comunicazioni:

```
interface TorreDiControllo {
    void inviaMessaggio(String msg, Aereo aereo);
}
class TorrePrincipale implements TorreDiControllo {
    List<Aereo> aerei;
    ...
}
abstract class Aereo {
    protected TorreDiControllo torre;
    public Aereo(TorreDiControllo t) { this.torre = t; }
    public abstract void ricevi(String msg);
    public abstract void invia(String msg);
}
class AereoPasseggeri extends Aereo { ... }
```

1. indicare il design pattern ed i nomi dei ruoli assegnati a **Torre** e **Aereo**,

2. completare **TorrePrincipale** affinché instradi il messaggio a tutti gli aerei tranne il mittente, e implementare i metodi in **AereoPasseggeri**,
3. implementare un **Client** che registra due aerei sulla torre e fa inviare un messaggio a uno di essi,
4. disegnare il diagramma UML delle classi fornite e create,
5. disegnare il diagramma UML di sequenza a partire dall'invio del messaggio nel **Client**.

2. Usando la programmazione funzionale di Java, data una lista **consegne** di istanze di **Pacco**, implementare una classe con:

```
public record Pacco(String tracking, String corriere, double pesoKg, String stato)
{ }
// 'stato' puo' essere "in transito", "consegnato", "perso"
```

1. un metodo che restituisce la lista dei **tracking** dei pacchi aventi **stato** uguale a "in transito",
2. un metodo che conta quanti corrieri diversi stanno effettuando consegne,
3. un metodo che restituisce la somma del peso (**pesoKg**) di tutti i pacchi,
4. un metodo che restituisce l'istanza di **Pacco** con il peso maggiore in assoluto.

Nome:

Matr.:

Prova 9

1. Per il seguente codice per la clonazione di oggetti:

```
interface Clonabile {
    Clonabile clona();
}
class Report implements Clonabile {
    private String titolo;
    private String dati;
    public Report(String t, String d) { ... }
    ...
}
```

1. indicare il design pattern che sfrutta interfacce simili per duplicare istanze, definendone i ruoli,
2. completare la classe **Report** aggiungendo l'implementazione corretta del metodo **clona()** in modo da restituire una copia profonda o superficiale (selezionarne una e indicarla a commento),
3. implementare una classe **Client** che istanzia un **Report** e ne produce una copia usando il metodo fornito dal pattern,
4. disegnare il diagramma UML delle classi fornite e della classe **Client**,
5. disegnare il diagramma UML di sequenza per la clonazione.

2. Usando la programmazione funzionale di Java, data una lista **squadra** di istanze di **Atleta**, implementare una classe con:

```
public record Atleta(String nome, String sport, int medaglie, int eta) { }
```

1. un metodo che trova gli atleti che hanno vinto almeno una medaglia e restituisce la lista dei loro nomi,
2. un metodo che restituisce la lista degli sport rappresentati in squadra, senza ripetizioni,
3. un metodo che restituisce la somma totale delle **medaglie** vinte da tutti gli atleti,
4. un metodo che restituisce l'atleta più giovane (minore **eta**).

Nome:

Matr.:

Prova 10

1. Per il seguente codice che fornisce un'interfaccia unificata:

```
class Proiettore { void accendi() { ... } void impostaInput() { ... } }
class Amplificatore { void accendi() { ... } void volume(int v) { ... } }
class LettoreDVD { void accendi() { ... } void play() { ... } }
class HomeTheaterFacade {
    Proiettore proiettore;
    Amplificatore ampli;
    LettoreDVD dvd;
    ...
}
```

1. indicare il design pattern ed i ruoli delle classi del sottosistema e della classe principale,
 2. completare **HomeTheaterFacade** aggiungendo il costruttore e un metodo **guardaFilm()** che accende tutti i componenti e fa partire il DVD,
 3. implementare un **Client** che utilizza il Facade appena configurato per guardare un film con una sola invocazione,
 4. disegnare il diagramma UML delle classi mostrando laFacade e le classi sottostanti,
 5. disegnare il diagramma UML di sequenza dell'invocazione di **guardaFilm()**.
2. Usando la programmazione funzionale di Java, data una lista **resoconto** di istanze di **Spesa**, implementare una classe con:

```
public record Spesa(String descrizione, String categoria, double importo, int mese) { }
```

1. un metodo che restituisce la lista delle **descrizione** per le spese effettuate in un **mese** specifico passato in input,
2. un metodo che conta il numero di categorie diverse registrate,
3. un metodo che restituisce la somma degli **importo** di tutte le spese della lista,
4. un metodo che restituisce l'istanza di **Spesa** con l'importo più alto in assoluto.